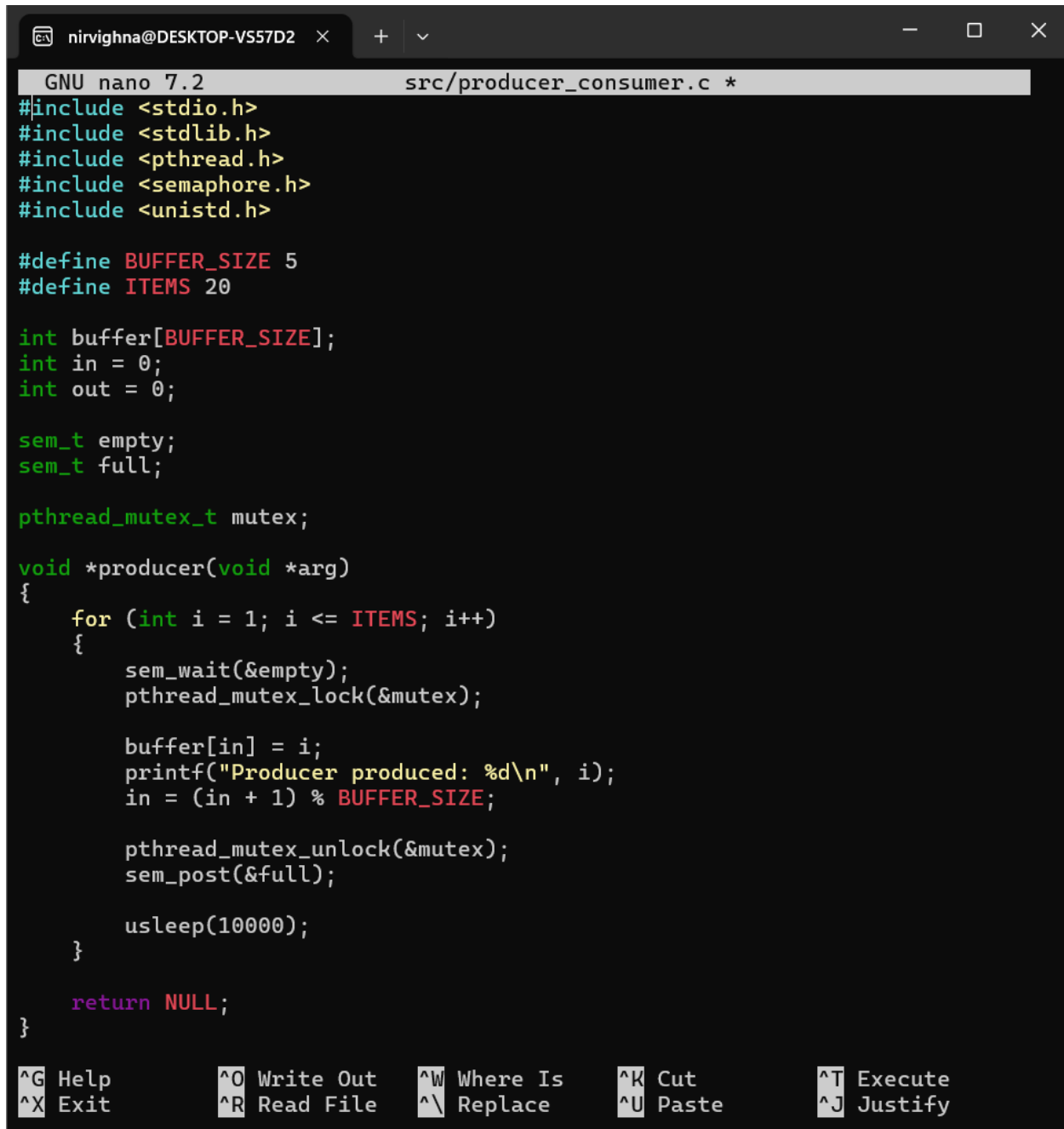


PRACTICAL 12:



```
GNU nano 7.2 src/producer_consumer.c *
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define ITEMS 20

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

sem_t empty;
sem_t full;

pthread_mutex_t mutex;

void *producer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++)
    {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        printf("Producer produced: %d\n", i);
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);

        usleep(10000);
    }

    return NULL;
}
```

 ^G Help	 ^O Write Out	 ^W Where Is	 ^K Cut	 ^T Execute
 ^X Exit	 ^R Read File	 ^N Replace	 ^U Paste	 ^J Justify

```
nirvighna@DESKTOP-VS57D2  ×  +  ∨  
Consumer consumed: 1  
Producer produced: 2  
Consumer consumed: 2  
Producer produced: 3  
Producer produced: 4  
Consumer consumed: 3  
Producer produced: 5  
Consumer consumed: 4  
Producer produced: 6  
Consumer consumed: 5  
Producer produced: 7  
Producer produced: 8  
Consumer consumed: 6  
Producer produced: 9  
Consumer consumed: 7  
Producer produced: 10  
Producer produced: 11  
Consumer consumed: 8  
Producer produced: 12  
Consumer consumed: 9  
Producer produced: 13  
Producer produced: 14  
Consumer consumed: 10  
Producer produced: 15  
Consumer consumed: 11  
Producer produced: 16  
Consumer consumed: 12  
Producer produced: 17  
Consumer consumed: 13  
Producer produced: 18  
Consumer consumed: 14  
Producer produced: 19  
Consumer consumed: 15  
Producer produced: 20  
Consumer consumed: 16  
Consumer consumed: 17  
Consumer consumed: 18  
Consumer consumed: 19  
Consumer consumed: 20  
Producer-Consumer execution completed.  
nirvighna@DESKTOP-VS57D2I:~/OS-Week1$ |
```

```
nirvighna@DESKTOP-VS57D2  ×  +  ▾
GNU nano 7.2 src/deadlock_demo.c *
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t resource1 = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t resource2 = PTHREAD_MUTEX_INITIALIZER;

void *thread1(void *arg)
{
    pthread_mutex_lock(&resource1);
    printf("Thread 1 locked Resource 1\n");

    sleep(1);

    pthread_mutex_lock(&resource2);
    printf("Thread 1 locked Resource 2\n");

    pthread_mutex_unlock(&resource2);
    pthread_mutex_unlock(&resource1);

    return NULL;
}

void *thread2(void *arg)
{
    pthread_mutex_lock(&resource2);
    printf("Thread 2 locked Resource 2\n");

    sleep(1);

    pthread_mutex_lock(&resource1);
    printf("Thread 2 locked Resource 1\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);

    return NULL;
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
```

```
nirvighna@DESKTOP-VS57D2I:~/OS-Week1$ ./bin/deadlock_demo
Thread 1 locked Resource 1
Thread 2 locked Resource 2
^C
nirvighna@DESKTOP-VS57D2I:~/OS-Week1$
```

```
nirvighna@DESKTOP-VS57D2I:~/OS-Week1$ Thread 1 locked Resource 1  
Thread 1 locked Resource 2  
Thread 1 completed  
Thread 2 locked Resource 1  
Thread 2 locked Resource 2  
Thread 2 completed
```